Key ceremony: repository root (YubiKey A) and release key (YubiKey B)

> **Status: OWNER DRAFT, revision 5, 2026-11-06.** Reviewed by Codex three times (`docs/reviews/ceremony/`: `CODEX-REVIEW.md` C01–C07, `CODEX-CONFIRM.md` F01–F05, `CODEX-CONFIRM-2.md`, `CODEX-CONFIRM-3.md`). **Not ready for the real ceremony until rehearsal 2 validates the raw touch readback in S8** (see "Rehearsal 2" at the end). Basis: `docs/V1-RELEASE-KEY-CUSTODY-DRAFT.md` §5–§8 and the first rehearsal (`2026-11-01-rehearsal-1.md`). Key type from `docs/V1-PROFILE.md` §15.4: v4 ECDSA P-256. The rehearsal showed a key made this way, on the author's Tails stick and a YubiKey 5, signs in a form the verifier's git path accepts, and that touch was enforced. It generates the keys only. Using B for a production release needs the custody draft's §9 (independent review, exact-hash approval, controlled input media) implemented and rehearsed first; generating B does not close A10.4.2's independence requirement (C07). **Secret** values (PINs, passphrases, secret keys, backup locations) never go on paper you photograph, into chat, the repository or the NUC. **Non-secret** choices and results (versions, serials, fingerprints, pass/fail) go into the custody record in the repository.

Why two keys, two sessions, separate backups

- **Repository root (A)** signs key-assertion commits and the end-of-issuance statement: the git channel's authority.
- **Release key (B)** signs verifier releases, which carry the trust file naming A. A relying party that installs a release authenticated by B trusts the repository keys it lists. So B must not be capturable together with A, the DNS credential or the AWS account (A10.4.2, registered).
- So each key gets its own session, passphrase, PINs and backup sticks, and A's and B's backups, papers and tokens are kept apart. Then finding one place yields at most one key's material. This does not defend shared hardware or boot firmware that is itself compromised (custody draft's peripheral assumption).

What you need (per key)

Item	Count	Notes
Tails boot stick	1 (shared)	**Persistent Storage must not be unlocked** in these sessions. Boot only.
YubiKey	1	A or B, labelled. C and D stay spares.
Backup sticks	2	New, never in the NUC. `ROOT-BK1`/`ROOT-BK2` or `REL-BK1`/`REL-BK2`.
Out stick	1	New. Public files only. Goes to the NUC afterwards and never returns.
Paper	2 sheets	Public sheet (versions, serial, fingerprint, hashes, pass/fail). Secret sheet (passphrase, PINs), kept apart from the backups.

Total new sticks for both keys: **6**.

Owner decisions (2026-11-02; recorded, not secret)

- **Hardware mode**: the key lives on the YubiKey; the two backups are the only other copies, and they are exportable signing authority, protected only by their passphrase and where they are kept.
- **Backups**: the passphrase-protected secret-key export on a LUKS2 volume made by Tails' *Disks*. One passphrase per key serves both layers; this is one barrier, not two, and is accepted. A's and B's passphrases are independent. The actual LUKS parameters are read back and recorded (step S5.4).
- **Card KDF off.** It would keep the PIN off the USB wire. Accepted boundary: the host and its USB path are trusted for the session. KDF would not help against a hostile host, which sees the PIN anyway.
- **No expiry** on either key. Validity comes from the trust configuration's signing interval and release lineage.
- **Touch fixed (`uif ... permanent`) and signature PIN forced.** If either cannot be confirmed, stop (step S8).
- **No reset code** on the card (retry counter shows 0 for it).

Before the first session (author)

1. Note the Tails version and where its image signature was verified (a device that is not the NUC). The author names the signing hardware (the

System76) and checks it for stored credentials and inbound access (custody draft §5): git/gh tokens, cloud CLIs, logged-in browsers, SSH server or `authorized_keys`. Record date and result.

2. Six new sticks, labelled.

Session 1: generate, back up, load the card (once per key, on separate boots)

Network cable out, Wi-Fi off. At the Tails Welcome Screen: **do not unlock Persistent Storage**; set an administration password (needed by *Disks*). Start Tails, open a terminal.

S0. Check the session. Every check must hold, or stop.

```
cat /etc/os-release | grep -E 'PRETTY|VERSION'; gpg --version | head -2; /usr/lib/gnupg/scda
ip -brief link                                          # every link except lo is DOWN or absen
swapon --show                                           # prints nothing
lsblk -o NAME,TYPE,MOUNTPOINT                           # no internal disk partition has a moun
export GNUPGHOME="$(mktemp -d /tmp/gnupg.XXXXXX)"; chmod 700 "$GNUPGHOME"; echo "$GNUPGHOME"
findmnt -n -o FSTYPE --target "$GNUPGHOME"              # must print: tmpfs
```

If the last line prints anything but `tmpfs`, stop: the procedure has not been checked on this Tails, and the owner must look before any key exists. (Run S0 alone once beforehand to find out; it creates nothing.) Tails does not run anything from inserted media; insert nothing but the sticks this procedure names, and open nothing on them except the files it names. Write the Tails, GnuPG and scdaemon versions and the `GNUPGHOME` path on the public sheet. Run nothing from the NUC: no script, no configuration file, no copied command file. **Every later command runs in this same terminal.** If you open another, type `export GNUPGHOME=<the path on the sheet>` (do not run `mktemp` again).

S1. Check the card. Insert only the labelled YubiKey.

```
gpg --card-status
```

Yubico, no keys, PIN retries `3 0 3`, a `UIF setting` line. Write the serial and the OpenPGP version (the `Version` line) on the label and the public sheet. (The YubiKey firmware number itself needs `ykman`, which this session does not have; OpenPGP 3.4 implies firmware 5.2.3 or later.)

S2. New PINs (never used elsewhere; on the secret sheet):

```
gpg --card-edit
admin
passwd
```

`3` admin PIN (default `12345678`), `1` user PIN (default `123456`), `q`, `q`. If a PIN is refused: stop, run `gpg --card-status`, read the retry counter. Do not guess.

S3. Signature slot to P-256 (expert mode is required for P-256 on some GnuPG versions):

```
gpg --expert --card-edit
admin
key-attr
```

Signature key: `2` (ECC), then the **NIST P-256** entry (its number may differ; read the list). For the encryption and authentication keys, choose the option that keeps the current attribute (or press Enter if offered). Admin PIN. `q`. Then `gpg --card-status`: Key attributes must begin `nistp256`.

S4. Generate in memory:

```
gpg --expert --full-generate-key
```

`10` (ECC, sign only), NIST P-256 entry, expiry `0`. Name `Tessera repository root` (A) or `Tessera release key` (B); email `tessera@wamason.com`. Passphrase. Then:

```
gpg --list-secret-keys --with-fingerprint
FPR=<the 40 hex digits, no spaces>
gpg --list-keys "$FPR" >/dev/null && echo FPR-OK
```

Write the fingerprint on the public sheet; read it back.

S5. Two backups, one stick at a time. 1. *Disks* → the stick → *Format Disk* (GPT) → `+` → *Internal disk for use with Linux systems only (Ext4)*, **Password protect volume (LUKS)**, passphrase, name `ROOT-BK1` (etc.). Unlock and mount it in *Files*. 2. Confirm it is the encrypted volume, then write and check each file (any `FAIL` → stop, see Failures):

```
B="/media/amnesia/ROOT-BK1"                          # exactly the name shown in Files
DEV="$(findmnt -n -o SOURCE --target "$B")"; MNT="$(findmnt -n -o TARGET --target "$B")"
[ "$MNT" = "$B" ] && [ "$(lsblk -no TYPE "$DEV")" = crypt ] && echo "LUKS-OK $DEV" || echo F
gpg --export-secret-keys --armor "$FPR" > "$B/secret.asc" && test -s "$B/secret.asc" && echo
gpg --export --armor "$FPR" > "$B/public.asc" && test -s "$B/public.asc" && echo PUBLIC-OK
cp "$GNUPGHOME/openpgp-revocs.d/$FPR.rev" "$B/revocation.rev" && test -s "$B/revocation.rev"
sync; ls -l "$B"
```

3. Record the LUKS parameters (not secret), from the partition under that exact mapper: `sudo cryptsetup luksDump "/dev/$(lsblk -no PKNAME "$DEV")" | grep -E 'Version|cipher|PBKDF'` must show `Version: 2`. Write it on the public sheet.
4. **Prove this backup restores, while the original key is still in memory** (a second, separate keyring):

```
T="$(mktemp -d /tmp/restore.XXXXXX)"; chmod 700 "$T"
GNUPGHOME="$T" gpg --batch --import "$B/secret.asc" && GNUPGHOME="$T" gpg --list-secret-keys
echo restore > "$T/r.txt"; GNUPGHOME="$T" gpg --local-user "$FPR" --digest-algo SHA256 --arm
gpgconf --homedir "$T" --kill gpg-agent; rm -rf "$T"
```

It asks for the passphrase (the backup's key passphrase). `FPR-MATCH` and `RESTORE-OK` both, or this backup is not good: redo S5 for it on a new stick. 5.

Eject in *Files*; remove.

S6. Public key to the out stick. A new out stick needs a plain (unencrypted) volume first: in *Disks*, select it → *Format Disk* (GPT) → `+` → *For use with all systems (FAT)*, no password, name `ROOT-OUT` (or `REL-OUT`). Mount it in *Files*, then check it is that exact mount before writing:

```
O="/media/amnesia/ROOT-OUT"
[ "$(findmnt -n -o TARGET --target "$O")" = "$O" ] && echo OUT-MOUNT-OK || echo FAIL-NOT-MOU
gpg --export --armor "$FPR" > "$O/root.pub.asc" && test -s "$O/root.pub.asc" && sha256sum "$
```

(`release.pub.asc` for B.) SHA-256 on the public sheet. Eject; remove. Nothing secret, and not the revocation certificate, ever goes on this stick.

S7. Move the key to the card (only after S5 printed every `-OK`, including `RESTORE-OK`, for both sticks):

```
gpg --edit-key "$FPR"
keytocard
```

Answer **y** to "Really move the primary key?", choose `1` (signature key), passphrase, admin PIN, then `save`. Then `gpg --card-status`: the signature key line shows your fingerprint. If not, see Failures (do not reset the card).

S8. Card policy, set and confirmed:

```
gpg --card-status | grep -E 'Signature PIN|UIF'
```

- If it says `Signature PIN ....: not forced`: `gpg --card-edit`, `admin`, `forcesig`, `q`. Re-read: must say `forced`. (`forcesig` toggles; never run it when it already says forced.)
- `gpg --card-edit`, `admin`, `uif 1 permanent`, `q`. Then read the card's raw value. GnuPG's own status output (human or `--with-colons`) reduces it to on/off and cannot show "permanent", so ask the card through the agent:

```
gpg-connect-agent 'SCD GETATTR UIF-1' /bye
```

Expected, exactly:

```
S UIF-1 %02+
OK
```

(`%02` is the policy byte, permanent; `+` is the second byte, 0x20, as the status line encodes it.) `%01+` is ordinary on and `%00+` is off: **stop**. An `ERR` line, no `S UIF-1` line, or anything else: **stop**. If `uif` was an unknown command: **stop**. Do not proceed without fixed touch. *(This readback is validated in rehearsal 2 before the real ceremony.)*

S9. Prove the policy, not just a signature: 1. Positive: sign, enter the user PIN, **touch when it blinks** (GnuPG does not prompt):

```
echo "tessera <root|release> ceremony $(date -u +%FT%TZ)" > ~/proof.txt
```

```
gpg --local-user "$FPR" --digest-algo SHA256 --armor --detach-sign ~/proof.txt && gpg --veri
```

2. No touch: enter the PIN, then **do not touch**; wait for it to fail.

```
rm -f ~/nt.asc; gpg --local-user "$FPR" --digest-algo SHA256 --armor --output ~/nt.asc --det
```

Pass: a timeout error, a nonzero exit and `NO-SIGNATURE-OK`. 3. Fresh PIN: sign again, touching this time. It must ask for the PIN again.

```
rm -f ~/p2.asc; gpg --local-user "$FPR" --digest-algo SHA256 --armor --output ~/p2.asc --det
```

Note whether the PIN was asked for again. 4. `gpg --card-status`: record the signature counter (1 per successful signature). Copy the proof to the out stick (the same one as S6, reinserted and rechecked; it has not left the session):

```
[ "$(findmnt -n -o TARGET --target "$O")" = "$O" ] && cp ~/proof.txt ~/proof.txt.asc "$O/" &
```

Eject it.

S10. Shut down. Remove the YubiKey.

Session 2: tests in fresh boots (no secret key in memory at start)

Each in its own boot, same S0 checks (a new `GNUPGHOME`), YubiKey **not** inserted unless stated. A new boot has no variables: set them by hand each time. 1. **Each backup** (twice: BK1, then BK2). Unlock only that stick in *Files*, then:

```
B="/media/amnesia/ROOT-BK2"                          # the stick actually inserted
FPR=<the fingerprint from the public sheet, no spaces>
gpg --batch --import "$B/secret.asc" && gpg --list-secret-keys --with-colons | grep "^fpr::
echo restore-test > ~/r.txt && gpg --local-user "$FPR" --digest-algo SHA256 --armor --detach
```

Record pass/fail on the public sheet. 2. **The card alone**: insert only the YubiKey and one backup stick (for its `public.asc` only; import nothing secret). Set the variables again: this is a new boot.

```
B="/media/amnesia/ROOT-BK1"                          # the stick actually inserted
FPR=<the fingerprint from the public sheet, no spaces>
gpg --import "$B/public.asc" && gpg --card-status >/dev/null
echo card-test > ~/c.txt && gpg --local-user "$FPR" --digest-algo SHA256 --armor --detach-si
```

PIN, then touch when it blinks. This proves the card works without any residue of the generation session.

Acceptance: S5 all `-OK`, S7 fingerprint on the card, S8 both policies confirmed, S9 steps 1–3 as expected, Session 2 all three pass. Anything else: the key is not accepted (see Failures).

Failures (do the safe thing first)

What happened	Do this
Any S0 check fails	Stop. Shut down. Nothing secret exists yet.
A backup step prints FAIL	The key is still in memory. Do not continue to S7. Reformat that stick (or use a new one) and redo S5 for it.
`keytocard` fails or the card shows no key after S7	Read `gpg --card-status`. Retry S3/S7 once. Shut down only if both backups printed `RESTORE-OK` in S5 step 4: then the key survives on proven copies, and goes onto another YubiKey in a later session. If fewer than two did, do not shut down: redo S5 on new sticks first. Do not reset the card blindly.
A PIN is refused	Stop; read the retry counter. Never guess down to zero.
S8 cannot confirm forced PIN or fixed touch	Stop before S9. The card holds the key without its policy; record it; decide with the owner.
S9 no-touch test signs anyway	Policy failed: the key is not accepted.
A Session 2 restore fails	That backup is not good. From the other backup or the card, nothing is lost: in a new session, restore the good backup and make a replacement on a new stick (S5), then retest.
Both backups fail in Session 2 but the card works	Keep the card; do not reset it. Stop and decide with the owner: accept a card-only key without backup (loss means a new key and a new release), or generate a new key. Nothing is published until decided.
Both backups and the card unusable	Generate a new key; the old fingerprint is discarded and never published.

After both keys

- **Storage (author):** each key's two backups in two different places; A's and B's material not together; secret sheets apart from backups. Tell the owner only "done".
- **Owner, on the NUC:** mount the out stick read-only; check each public key with the verifier's code (82-byte v4 P-256 body) and the proof signatures; write the dated custody record: Tails and GnuPG versions, image verification device, signing hardware and its credential check, YubiKey

serials, fingerprints, LUKS parameters, card policies read back, test results, this revision's hash, the rehearsal link, "backups in two places: yes". No secrets, no locations.

- **Independent fingerprint (author):** on a device that is not the NUC, compute the fingerprint from the public key file itself (not from a printed string) and compare with the paper.
- **Retained outside the NUC:** each backup stick holds `public.asc`, so the approved public keys exist away from the operational environment.
- A's public key then enters the trust configuration's `repository_keys`.
- **B's first acquisition (needed before B's first production use, not for this ceremony):** two matching web publications do not authenticate a key; an attacker holding the repository and DNS could substitute both. The author names one channel that passes through neither the NUC, the repository nor DNS (for example a fingerprint the author publishes on a university page, or prints in a signed paper), and after publication checks the actual published fingerprint in each place against the paper. The dated first-production-release bootstrap record links B's fingerprint to the freeze baseline (A10.4.2).

Time

Estimate per key: 45–60 minutes for Session 1, about 10 minutes per test boot in Session 2 (three per key). The first real session will measure it.

Revision 2 changes (Codex C01–C07)

C01 `--expert --card-edit` and read the curve list. C02 versions recorded at S0; touch confirmed by readback and the no-touch test, stop if unconfirmed (the rehearsal's `uif` was accepted and touch was enforced, so the author's Tails evidently has a GnuPG newer than 2.2.40). C03 no Persistent Storage, RAM `GNUPGHOME`, session checks, credential check on the hardware. C04 checked exports, LUKS confirmation, acceptance conditions, failure table. C05 `forcesig` read before toggling; no-touch and fresh-PIN tests; card-only test in a fresh boot. C06 non-secret choices recorded. C07 B's production use waits for §9; independence of requesters is not claimed here.

Revision 3 changes (Codex F01–F05)

F01 *(superseded by revision 4: the colon output cannot show permanent)* touch read back with `--with-colons`. F02 each backup is restored and used in a separate keyring while the original key is still in memory (S5 step 4); the card transfer and every shutdown in the failure table depend on two proven backups; a working card is never reset because backups failed. F03 Session 2 sets `B` and `FPR` by hand and gives complete commands. F04 the destination is checked by its exact mount and mapper, and LUKS2 is read from the partition under that mapper. F05 B's first acquisition needs a channel outside the NUC, repository

and DNS, chosen by the author before B's first production use; public keys are retained on the backup sticks. C02/C03 residuals: scdaemon version recorded; GNUPGHOME must be on tmpfs or stop; no NUC scripts.

Rehearsal 2 (before the real ceremony): a dress rehearsal of this revision on YubiKey C

YubiKey C was factory-reset after rehearsal 1, so rehearsal 2 runs the whole procedure with a throwaway key: Session 1 (S0–S10) and Session 2, exactly as written, with these substitutions: - Name the key `Tessera REHEARSAL 2`; YubiKey **C**; throwaway PINs and passphrase (still never photographed). - Backups: the two sticks that will be **A's** backups (one Kingston, one Lexar). The real ceremony reformats them in S5, which destroys the throwaway backup. - Out stick: the new `ROOT-OUT` (Kingston), **used offline only**: it never goes to the NUC during or after the rehearsal, and the real ceremony reformats it in S6. **Never the rehearsal-1 stick**: it touched the NUC and must not enter a signing session (the error in revision 4, Codex H01). - At S8, record the output of `gpg-connect-agent 'SCD GETATTR UIF-1' /bye` (a photo is fine). **This is the check rehearsal 2 exists for:** the first byte must be `02`. If not, stop and bring the output to the owner. - After Session 2: factory-reset C (`gpg --card-edit`, `admin`, `factory-reset`) and shut down. Nothing goes to the NUC. Photograph (public outputs only): the S0 versions, the S8 `SCD GETATTR UIF-1` output, the S9 results (`NO-SIGNATURE-OK`, `P2-OK`) and the Session 2 results. Note step times and anything that did not go as written. The verifier's acceptance of this kind of signature was already shown in rehearsal 1.

Sticks (author, 2026-11-06): 3 Kingston, 3 Lexar

Role	Vendor
`ROOT-BK1`	Kingston
`ROOT-BK2`	Lexar
`REL-BK1`	Lexar
`REL-BK2`	Kingston
Out stick, root session	Kingston
Out stick, release session	Lexar

Each key's two backups are from different vendors, which reduces the chance that one bad batch takes both copies (it does not rule out a shared component). Label them now.

Revision 4 changes (Codex confirmation 2)

F01: Codex withdrew its own revision-2 advice; GnuPG's status parser reduces UIF to on/off, so S8 now reads the raw value from the card via `SCD GETATTR`

`UIF-1`, to be validated in rehearsal 2. F03: the card-only boot sets `B` and `FPR`. C03: no-autorun stated. C02/C06: OpenPGP version recorded; firmware needs `ykman`, not available offline.

Revision 5 changes (Codex confirmation 3)

H01: rehearsal 2 no longer uses the NUC-exposed rehearsal-1 stick; it uses the new `ROOT-OUT` offline only, and nothing from the rehearsal goes to the NUC. F01: expected output corrected to `S UIF-1 %02+`, with explicit stop conditions. Out stick: formatted, labelled and checked by exact mount at S6; S9 copies to it with a printed command. S9: full commands for the no-touch and fresh-PIN tests. Vendor wording bounded. Revision 3's colon-readback summary marked superseded.